Memory-Augmented SmolVLA — Final Report

memvla_libero: a MemoryVLA-port to SmolVLA

tarcode2004 / aleksantari

May 2026

Contents

0.1 1. Executive Summary . . . 3
0.2 2. Background and Motivation . . . 4
0.2.1 2.1 VLAs are memoryless . . . 4
0.2.2 2.2 LIBERO long-horizon as the natural target . . . 5
0.2.3 2.3 The two systems we sit between . . . 5
0.3 3. SmolVLA Architecture (Reference) . . . 6
0.3.1 3.1 Components . . . 6
0.3.2 3.2 Eight cross-attention pathways — where memory can enter . . . 6
0.3.3 3.3 The truncation — why we don't have a "cognitive" stream . . . 6
0.4 4. Three Architectural Asymmetries . . . 7
0.4.1 4.1 Asymmetry 01 — Representation: no summary token . . . 7
0.4.2 4.2 Asymmetry 02 — Injection: 8 cross-attention reads . . . 7
0.4.3 4.3 Asymmetry 03 — Data: contiguous windows vs sparse sampling . . . 7
0.5 5. memvla_libero — implementation . . . 9
0.5.1 5.1 Memory module architecture . . . 9
0.5.2 5.2 Why memory enters at L16 with `inject_before=True` . . . 10
0.5.3 5.3 Training regime . . . 12
0.5.4 5.4 Eval protocol . . . 13
0.6 6. Results . . . 14
0.6.1 6.1 Headline (10 ep $\times$ 10 task $\times$ 4 suites = 400 episodes) . . . 14
0.6.2 6.2 The gate is healthy — memory is being used, not ignored . . . 14
0.6.3 6.3 Per-suite cost correlates with rollout length . . . 14
0.7 7. Higher Batch Diversity Rerun — Results (memvla v2) . . . 15
0.7.1 7.1 What this config tests . . . 15
0.7.2 7.2 Results — 65K steps (run still in progress, planned 100K not reached due to time) 16
0.7.3 7.3 What this means for the hypotheses . . . 16
0.7.4 7.4 Caveats . . . 17
0.7.5 7.5 Same steps $\neq$ same data — the comparison-axis caveat . . . 17
0.7.6 7.6 Episode-diversity ladder and the SmolVLA convergence floor . . . 17
0.8 8. Discussion — Three Hypotheses . . . 19
0.8.1 8.1 H1 — Signal dilution . . . 19
0.8.2 8.2 H2 — Distribution mismatch (train bank $\neq$ eval bank) . . . 19
0.8.3 8.3 H3 — Storage cost (no CLS $\rightarrow$ expensive, coarse memory) . . . 20
0.8.4 8.4 Which hypothesis the data most cleanly supports . . . 20
0.8.5 8.5 Hypothesis status . . . 21
0.9 9. Future Directions . . . 22
0.9.1 9.1 Finish the diversity run to 100K and re-eval . . . 22
0.9.2 9.2 Bypass eval on diversity-65K (or 100K) checkpoint . . . 22

0.9.3 9.3 Push diversity higher 22
0.9.4 9.4 (addresses H2 residual) MemoryVLA-style sparse-frame sampling 22
0.9.5 9.5 (addresses H1) Route every cross-attn read through memory 23
0.9.6 9.6 (addresses H3) Learned summary token (synthetic CLS) 23
0.9.7 9.7 Architecture-agnostic findings from earlier iterations 23
0.10 10. References 24

0.1 1. Executive Summary

This report documents the design, implementation, and evaluation of a temporal memory module added to the SmolVLA vision-language-action policy. The production model — `memvla_libero` — is a port of MemoryVLA's `CogMemBank` to SmolVLA's compact (450M) architecture, with one architectural adaptation forced by SmolVLA's fused interleaved VLM-and-expert layout.

The model was evaluated on the LIBERO benchmark (4 suites $\times$ 10 tasks $\times$ 10 episodes = **400 rollouts**) under two training configurations:

Configuration	spatial	object	goal	libero_10	**Overall**
V2	**84.0**	**99.0**	**96.0**	**72.0**	**87.75**
baseline (no memory, 100K steps)					
memvla v1 — (`num_groups=4, group_size=8`) @ 100K	74.0	96.0	79.0	44.0	73.25
memvla v1 **bypass** (gate forced α=1, same checkpoint)	72.0	96.0	82.0	54.0	76.00
memvla v2 — diversity rerun (`num_groups=12, group_size=4`) @ **65K** *(in progress)*	**84.0**	**97.0**	**94.0**	**52.0**	**81.75**

The headline result is the **diversity rerun**. Same architecture, same `mem_length=4`, **zero ToMe consolidations during training** (because `group_size = mem_length = 4`), but with **12 distinct episodes per gradient step** instead of 4. Even at 65K of a planned 100K steps:

- **`libero_spatial` matches baseline exactly** (84.0 / 84.0). `libero_object` and `libero_goal` are within 2pp of baseline.
- **+8.5pp overall** vs the original memvla v1 — same architecture, only `(num_groups, group_size)` changed.
- Gate stays active at $\alpha \approx 0.504$ — memory continues to be used and is now actually helping.
- **`libero_10` is the residual gap**: 52.0 vs baseline 72.0 (—20pp). Improved +8pp vs v1 (44 $\rightarrow$ 52) but not closed.

Combined with v1's bypass ablation, the data point to a **gradient-diversity bottleneck** as the dominant cause of v1's failure: 4 distinct episodes per batch was insufficient for stable convergence of a memory-conditioned expert. The original "memory hurts" reading was a downstream symptom of under-training, not a property of the architecture itself. The remaining `libero_10` gap is consistent with secondary effects (under-training — v2 is still climbing at 65K, plus the contiguous-vs-sparse training-bank asymmetry of §4.3) and is the next experiment to close.

The hypothesis "a temporal memory bank can be added to a frame-by-frame VLA without breaking it" is **supported** by v2; the stronger claim "memory improves long-horizon performance" remains **open** — the residual `libero_10` gap may close at 100K and/or with §9.4's sparse-sampling fix.

0.2 2. Background and Motivation

0.2.1 2.1 VLAs are memoryless

Vision-language-action models like SmolVLA are **Markovian**: at every frame, the policy re-runs `image + text` → `VLM` → `action` with no episodic context. Hidden state `h_t` is computed from `img_t` alone — the model does not know what it already did in the current rollout.

This is fine for pick-and-place. It breaks on long-horizon tasks — *open drawer, place item, close drawer* — where the right action depends on what already happened. The policy cannot tell whether it has cycled the gripper, opened the right container, or placed the correct object.

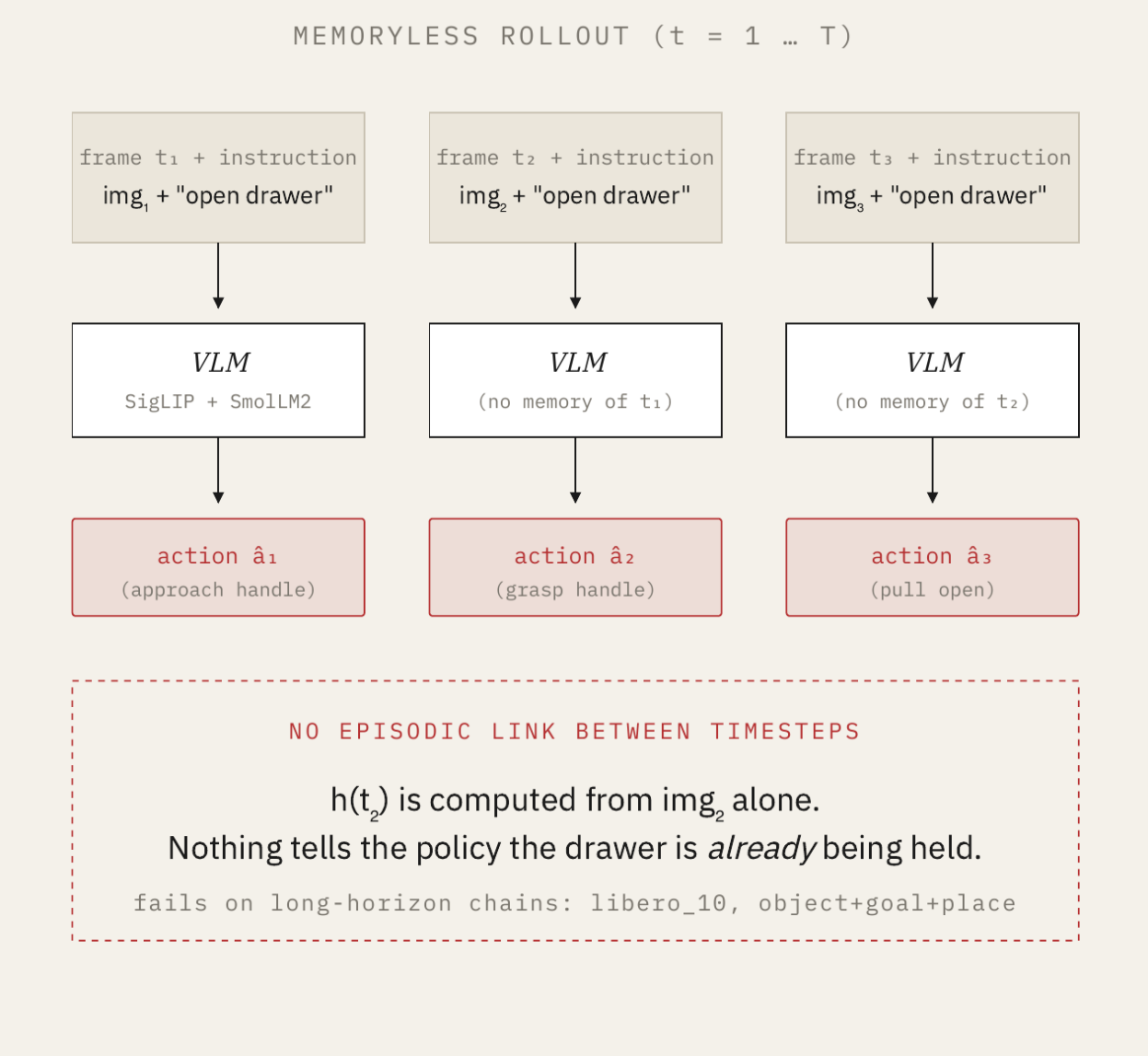

Figure 1: Three timesteps of a memoryless VLA rollout. Each frame is processed independently — there is no episodic link between timesteps. `h_t` is computed from `img_t` alone, so the policy cannot tell that the drawer is already being held.

0.2.2 2.2 LIBERO long-horizon as the natural target

LIBERO's `libero_10` suite is the long-horizon benchmark (10-step task chains, episodes up to 520 frames). Our baseline SmolVLA reproduction (RTX 5090, 100K steps, `n_action_steps=10`) shows where the headroom lies:

Suite	Task type	V2 baseline	Paper Table 13	Headroom
libero_spatial	Single-step pick-and-place	84.0	89	~16pp
libero_object	Single-step picking	99.0	94	~1pp
libero_goal	Goal-conditioned single-step	96.0	91	~4pp
libero_10	**10-step long-horizon**	**72.0**	**57**	**~28pp**
Overall		**87.75**	**82.8**	—

Object and goal are saturated. `libero_10` has ~28pp of headroom — by far the most. Memory-augmented policies *should* help most here.

0.2.3 2.3 The two systems we sit between

Our project sits between SmolVLA (the base) and MemoryVLA (the memory reference):

- **SmolVLA** (arXiv:2506.01844) — a 450M VLA with a frozen VLM and a flow-matching action expert that cross-attends to VLM hidden states at every even VLM layer (L2, L4, …, L16). Compact, on-device viable. Our starting point.
- **MemoryVLA** (arXiv:2508.19236) — adds a cognitive memory bank of CLS-token summaries between the VLM and the action head, consolidated via ToMe (token-merge). +26pp on long-horizon real-world manipulation vs CogACT base. Built on OpenVLA. Assumes a CLS-style summary token exists. Our inspiration.

Our project: port MemoryVLA's idea into SmolVLA. SmolVLA's architecture forces three major adaptations. Those adaptations — and what they taught us — are most of this report.

0.3 3. SmolVLA Architecture (Reference)

0.3.1 3.1 Components

Component	Specs
Vision encoder (SigLIP)	512×512 → 64 tokens/image @ 576 dim, 2 cameras
Language model (SmolVLM2-500M's text model)	**32 transformer layers natively**; SmolVLA truncates to the first 16 via `text_model.layers[:16]`
Action expert (`lm_expert`)	Flow-matching transformer paired 1:1 with the VLM (16 expert layers); 50-step action chunks; 10 denoising steps
Prefix structure	2× [start, 64 img tokens, end] + ~30–50 language tokens + 1 state token = ~163–183 tokens

0.3.2 3.2 Eight cross-attention pathways — where memory can enter

The action expert is paired 1:1 with the VLM (16 expert layers, 16 VLM layers). With `attention_mode="cross_attn"` and `self_attn_every_n_layers=2`, the layer loop alternates self-attention and cross-attention. Half the layers are cross-attention layers where the expert reads VLM K,V:

1-indexed layer (deck convention)	0-indexed (code)	Op type
L1, L3, L5, L7, L9, L11, L13, L15	0, 2, 4, 6, 8, 10, 12, 14	self-attention only
L2, L4, L6, L8, L10, L12, L14, L16	**1, 3, 5, 7, 9, 11, 13, 15**	**cross-attention (expert reads VLM K,V at this layer)**

So the action expert reads VLM hidden states at **every other VLM layer — 8 cross-attn handoffs**, not just at the final output. **There is no single post-VLM seam** where memory could be inserted to influence the entire downstream expert pass. The VLM and expert are interleaved.

(Throughout this report we use the deck's 1-indexed convention. Where it matters for code traceability, the 0-indexed value appears in parentheses.)

0.3.3 3.3 The truncation — why we don't have a "cognitive" stream

SmolVLA discards layers 17–32 of SmolVLM2's text model. The upper half of an autoregressive multimodal LLM is where semantic abstraction concentrates; with those layers gone, no part of the prefix at any reachable injection point is deeply semantic. MemoryVLA's two-stream design (perceptual + cognitive, with the cognitive token coming from LLaMA-7B's full layer-32 EOS hidden state) has no analog in our pipeline. We use a **single-stream full-sequence bank** instead — this is one of three structural adaptations forced by SmolVLA's architecture (§4).

0.4 4. Three Architectural Asymmetries

Three asymmetries between MemoryVLA's setup and ours dictate the design space. Each has a concrete consequence for `memvla_libero`.

0.4.1 4.1 Asymmetry 01 — Representation: no summary token

Open VLMs (OpenVLA's LLaMA-7B etc.) expose a CLS or EOS token that summarises the whole prefix sequence in a single position. **SmolVLM has none.** The truncation to layer 16 (§3.3) means even at the deepest reachable point, no token aggregates scene+task into a single summary.

Consequence: store the **full L-token sequence per bank entry**. Bank cost scales **L**$\times$ in keys at retrieval (L $\approx$ 170 tokens for our 2-camera LIBERO setup). With `mem_length=4` entries this caps at ~680 keys, vs MemoryVLA's 4 keys.

0.4.2 4.2 Asymmetry 02 — Injection: 8 cross-attention reads

The action expert cross-attends to VLM K,V at every even operative layer — L2, L4, L6, L8, L10, L12, L14, L16. MemoryVLA's reference architecture has a **single** VLM$\rightarrow$expert pathway. Ours has **eight**.

Consequence: memory injection at any single layer **structurally dilutes the signal 1/8**. Augment one pathway (we choose L16, §5.2), and the other seven reads stay vanilla — the expert is anchored to the un-augmented representation by 7/8 of its cross-attn handoffs. This is a deliberate accepted trade-off: single-point, minimally invasive modification first. Multi-layer injection is the obvious next ablation (§9.5).

0.4.3 4.3 Asymmetry 03 — Data: contiguous windows vs sparse sampling

Both MemoryVLA and our memvla_libero need a temporal bank built across timesteps within an episode, so neither can sample frames Markovianly during training. But the *form* of group sampling differs structurally — and the difference is the source of H2 (§8.2).

MemoryVLA's training sampler. Verified from `vla/datasets/rlds/dataset.py:716–730`: from a T-frame trajectory, *randomly subsample* `group_size`=16 frame indices, then sort them by timestep. The 16 frames span the whole episode in temporal order but are **non-contiguous** — typically ~T/16 apart on a long episode.

```
def sample_case():                # when episode has ≥ group_size frames
    shuffled = tf.random.shuffle(tf.range(T))
    return tf.sort(shuffled[:group_size])
```

Our `GroupedEpisodeLoader` (`src/memory_smolvla/data/group_loader.py`): yields `group_size`=8 *contiguous* frames at a random window offset within the episode.

The downstream effect on bank-state distribution at training time is the central difference:

	MemoryVLA on LIBERO	memvla_libero (ours)
`group_size`	16	8
Sampling within trajectory	random subsample $\rightarrow$ sort	contiguous window
Bank-entry temporal span at training	full episode (~500 frames)	8 consecutive frames
`mem_length`	16	4
Train-time consolidations per group	0 (`group_size` = `mem_length`)	4
Bank-entry temporal span at eval	full episode	full episode (520 max)
Train $\leftrightarrow$ eval span match	**yes**	**no — ~65$\times$ gap**

Consequence: MemoryVLA's training bank already spans the whole episode (just sparsely sampled). The expert sees temporally-diverse bank entries during training, and at eval the bank also spans the whole episode (densely after many ToMe merges). The two distributions are aligned. *They can get away with `group_size` = `mem_length` (zero train-time consolidations) precisely because their sparse sampling does the temporal-diversity work.*

Our 8 contiguous frames produce nearly-identical bank entries (consecutive frames share most of their content), so the expert is trained on homogeneous banks and sees heterogeneous banks at eval. This is the precise mechanistic statement of H2 (§8.2). The most direct fix is to switch our loader to MemoryVLA-style sparse-then-sort sampling — see §9.4.

0.5 5. memvla_libero — implementation

0.5.1 5.1 Memory module architecture

Implementation: `src/memory_smolvla/memory/full_seq_bank.py` and `src/memory_smolvla/memory/blocks.py` (on `dev` branch).

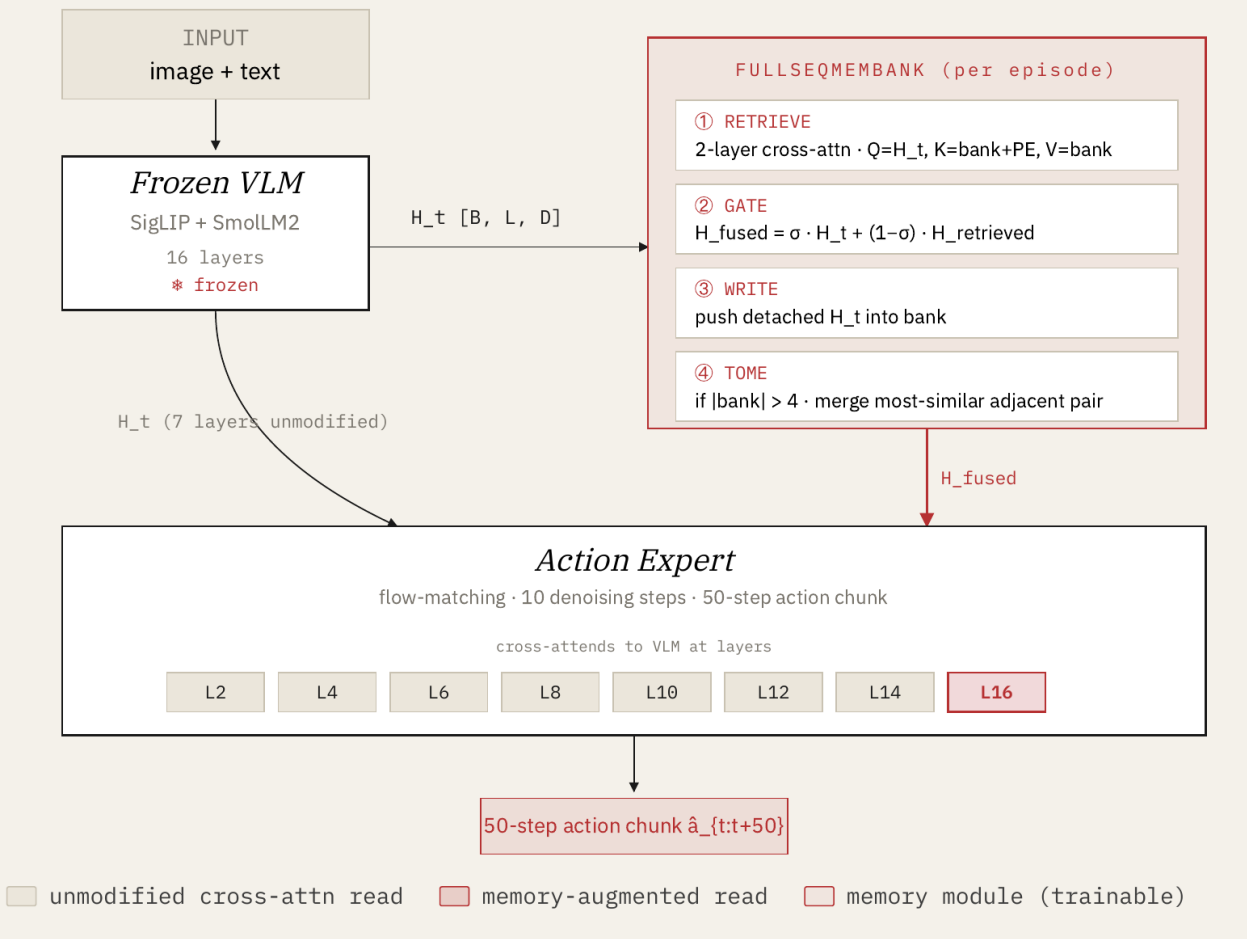

Figure 2: FullSeqMemBank architecture and design decisions. The memory module sits between the frozen VLM and the action expert: (1) RETRIEVE via 2-layer cross-attention with H_t as query and the bank (with sinusoidal timestep PE added to keys) as K/V; (2) GATE blends current and retrieved features with a per-token sigmoid; (3) WRITE pushes detached H_t into the bank; (4) ToMe merges the most-similar adjacent pair when the bank exceeds 4 entries. Memory is read by the action expert only at L16 — the other 7 cross-attention reads see unmodified H_t.

Component	Setting
Bank	per-episode dict; each entry stores the full prefix sequence `[L, D]`, keyed by `(timestep, episode_id)`
Capacity (`mem_length`)	**4** entries (chosen so consolidation fires during training — see §5.3)
Consolidation	**ToMe (Token Merging)** — when bank exceeds `mem_length`, merge the adjacent pair with highest cosine similarity (`@torch.no_grad()`)

Component	Setting
Retrieval	**2 stacked `CrossTransformerBlocks`** — single-head SDPA, post-norm, 4× FFN
Temporal PE	`TimestepEmbedder`: sinusoidal embedding → 2-layer MLP, **added to bank keys only** (not values)
Gate	**`GateFusion`** — single `Linear(2D, D)` + sigmoid; per-token, per-channel scale $\alpha \in$ [0,1]^D
Gate init	`std`=1e−3 on both weight and bias (matches MemoryVLA — *not* zero-init) → at init, $\alpha \approx 0.5$ per token
Gate convention	α=1 means all current (memory ignored); α=0 means all retrieved. Bypass ablation forces α=1.
Cold-bank path	when bank empty, `retrieved := current` so `gate_fusion(current, current) = current` — verified by `tests/test_full_seq_bank.py::test_cold_bank` (`atol`=1e−5)

0.5.2 5.2 Why memory enters at L16 with `inject_before=True`

`memvla_libero` injects memory at **L16 with `inject_before=True`** — on the residual-stream tensor (un-normalized) between L15's residual write and L16's `input_layernorm`. In the deck's 1-indexed convention this is "the layer-16 decision"; in 0-indexed code it is `injection_layer: 15, inject_before: true`.

Three reasons fix this choice:

1. **No clean post-VLM seam.** §4.2 established that the expert reads VLM K,V at every other VLM layer. There is no single boundary where memory could be inserted to influence the entire downstream expert pass.
2. **L16 is the expert's *final* read of VLM features.** Injecting earlier (e.g., L8) would let downstream VLM layers wash out the memory signal through up to 14 further layers of self-attention and MLP transformation before the expert's last cross-attn could use it. At L16, memory-fused features enter the K,V cache that the expert's last and highest-level cross-attn reads from.
3. **`inject_before=True` puts memory in the right place for L16.** It modifies the residual stream *before* L16's layer-norm, so both L16's VLM self-attn (which builds the K,V cache) and the L16 expert cross-attn (which reads it) operate on the memory-fused state. With `inject_before=False` at L15, only later layers would see memory; with `inject_before=True` at L16, the *current* layer's K,V cache is what gets modified.

Trade-off accepted: 1/8 signal dilution. The expert's cross-attns at L2, L4, L6, L8, L10, L12, L14 fire before the injection and read un-augmented VLM features. Only L16's cross-attn reads memory-fused K,V. This is a deliberate single-point, minimally invasive modification. Routing every cross-attn read through memory is the obvious next ablation (§9.5) — it directly tests H1.

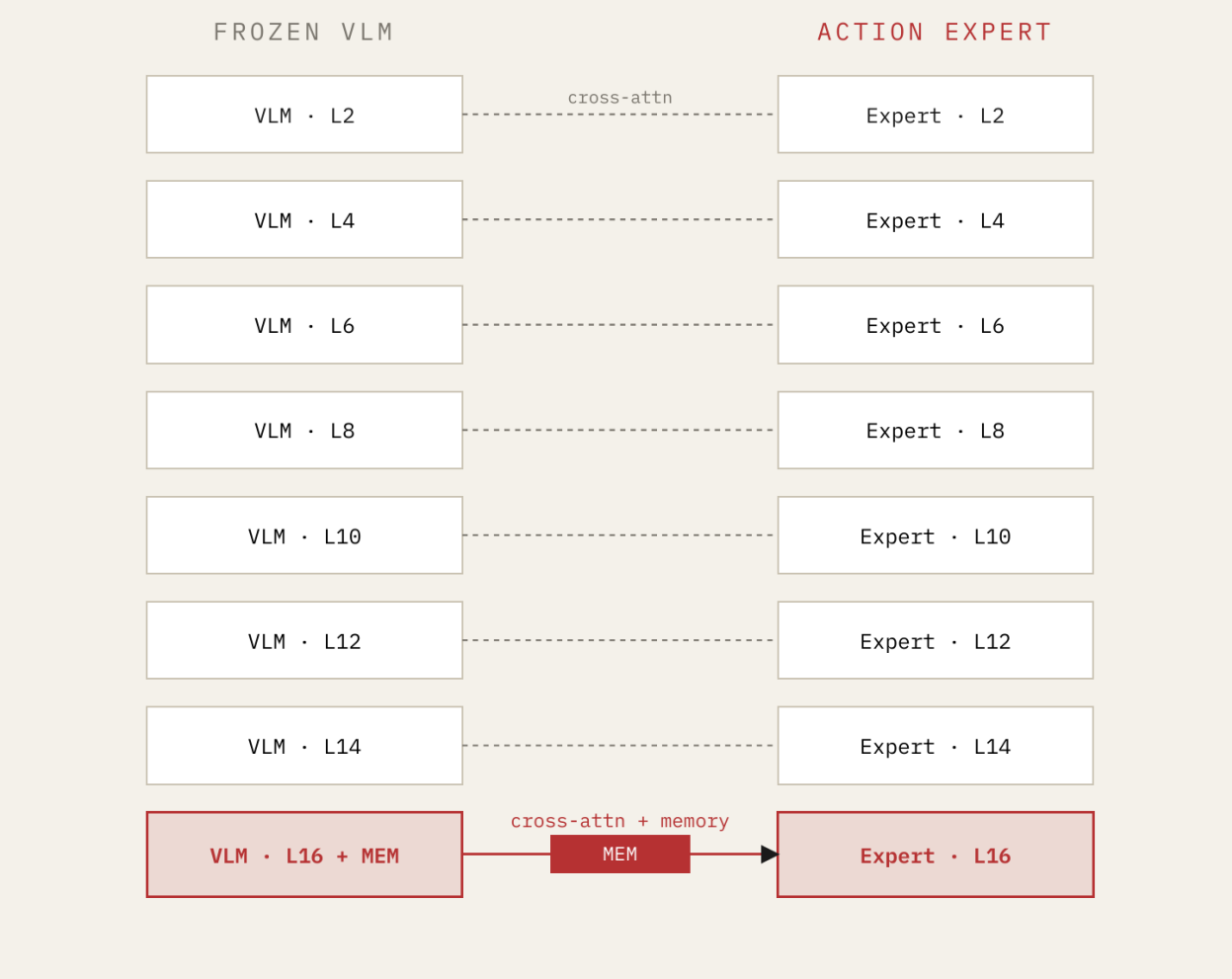

Figure 3: Eight cross-attention reads between VLM and expert. Memory is injected once, at L16, before the expert's *final* read of VLM features. The other seven cross-attention reads (L2–L14) see unmodified VLM features and anchor the expert to the vanilla representation. We accept this 1/8 signal dilution in exchange for a single-point, minimally invasive modification.

0.5.3 5.3 Training regime

0.5.3.1 Hyperparameters All hyperparameters mirror the V2 baseline run for apples-to-apples comparison.

Parameter	Value
Base checkpoint	`lerobot/smolvla_base` (VLM weights only; action expert reinitialized)
Batch size	**32** = `num_groups=4` × `group_size=8`
Total steps	100,000
Optimizer	AdamW
Peak LR	1e-4 (memory + expert)
Schedule	cosine decay to 2.5e-6 over 100K steps, 1K warmup
Weight decay	1e-10
Max grad norm	10.0
AMP	`bfloat16` autocast (no GradScaler)
`n_action_steps`	10 (paper Table 13 default)
Image augmentations	ColorJitter + SharpnessJitter + RandomAffine

0.5.3.2 Trainable parameter budget

Submodule	Total	Trainable
`vlm_backbone_frozen` (VLM text + vision)	350.2M	0
`action_expert_scratch` (lm_expert + action_out_proj, **reinitialized**)	98.3M	98.3M
`memory_scratch` (FullSeqMemBank: retrieval blocks + gate + timestep encoder)	23.3M	23.3M
`other` (proj heads, norms attached to base policy)	1.6M	0
Total	**473.3M**	**121.6M** (25.7%)

The action expert is **reinitialized from scratch** after loading `lerobot/smolvla_base` (only VLM + SigLIP weights are kept). This matches V2 baseline's from-scratch action-expert protocol and keeps the training comparison apples-to-apples.

0.5.3.3 The `mem_length=4`, `group_size=8` choice — and the design space it sits in `mem_length=4` is an internal choice, not a parity constraint. The hard constraint is **`group_size` $\geq$ `mem_length`**: otherwise the bank never fills during training and ToMe consolidation never fires. With `mem_length=4`, `group_size=8`, the bank fills at the 4th group frame and consolidates at frames 5, 6, 7, 8 — **4 consolidations per group**. This *does* exercise the consolidation path during training, though §6 will show that 4 consolidations is far below eval-time depth.

Under batch-32 parity, the design space is:

num_groups	group_size	max mem_length	Episodes per batch	Train-time consolidations / group
32	1	1	32	0 (bank never fills)
8	4	4	8	0
4	**8**	**4** *(used)*	**4**	**4**
2	16	16	2	up to 12
1	32	32	1	up to 28

The current config sits at 4 episodes per batch — a middle ground between gradient diversity (more episodes per batch is better) and consolidation depth during training (deeper consolidation per group is better). §7 revisits this trade-off with a 12-episode-per-batch rerun.

0.5.4 5.4 Eval protocol

`scripts/eval_memory_libero.py`: per-episode `LiberoEnv` instantiation (each of 10 episodes per task uses a different init state), image rotation via `_format_raw_obs`, env-specified `_max_episode_steps` per suite (280 / 280 / 300 / **520 for libero_10**), `start_seed + ep` per-episode seeding, `policy.reset()` at rollout start (clears action queue + memory bank), `policy.eval()` before rollouts. **10 episodes/task $\times$ 10 tasks $\times$ 4 suites = 400 rollouts per checkpoint.**

The bypass ablation is run on the *same checkpoint* with `mem_bank.bypass = True` — short-circuits retrieval and fusion, returning current tokens unchanged (equivalent to forcing α=1). Same seeds, same envs, same 400 rollouts. The bypass run isolates "what would this trained expert do without memory at inference" from "what does memory contribute on top of the expert."

0.6 6. Results

0.6.1 6.1 Headline (10 ep × 10 task × 4 suites = 400 episodes)

Source: `results/sim_memory/all_memvla_libero.json` (memory ON), `results/sim_memory/all_memvla_libero_` (bypass).

Suite	V2 baseline	memvla **bypass**	memvla **mem-on**	Δ (mem-on — bypass)	Δ (mem-on — baseline)
libero_spatial	**84.0**	72.0	74.0	+2.0	—10.0
libero_object	**99.0**	96.0	96.0	0.0	—3.0
libero_goal	**96.0**	82.0	79.0	—3.0	—17.0
libero_10	**72.0**	54.0	**44.0**	**—10.0**	**—28.0**
Overall	**87.75**	**76.00**	**73.25**	**—2.75**	**—14.50**

Two distinct gaps:

- **Gap A — training-mode regression (—11.75pp).** Even with memory bypassed, the memvla checkpoint underperforms V2 baseline by 11.75pp. The training regime is the same (batch 32, 100K steps, baseline-v2 hyperparameters) — what's different is that this checkpoint trained an action expert that *expects memory perturbations at L16* and has co-adapted to them. With memory bypassed at eval, the expert sees a cleaner signal than it was trained on, but it's still not the fully-finetuned-on-LIBERO expert that V2 baseline produced. This 11.75pp gap is the cost of joint memory + expert training itself.
- **Gap B — memory cost (—2.75pp).** The marginal cost of turning memory on, after Gap A is paid. Concentrated entirely on `libero_10`: 54 → 44 = **—10pp on the suite memory was supposed to help most**.

0.6.2 6.2 The gate is healthy — memory is being used, not ignored

The training and eval gate-value statistics are unambiguous:

- **Gate trained healthily.** `avg_gate_value` $\alpha \approx 0.49$ throughout training and eval — never collapsed to 1 (memory ignored), never saturated at 0 (memory dominating). Per-token gate variance is non-trivial.
- **Memory is actively read.** $\alpha \approx 0.49$ means the fused features are a near-equal blend of current and retrieved. The expert is not learning to gate memory off; it is learning to integrate it.
- **And the integration hurts.** On long rollouts, the actively-integrated memory is a net drag.

This is not a failed-training story. The model successfully learned to use memory — and the use of memory is what costs us 2.75pp overall.

0.6.3 6.3 Per-suite cost correlates with rollout length

Suite	max_steps	Δ memory cost (mem-on — bypass)
libero_spatial	280	+2 (mild help)
libero_object	280	0 (saturated)
libero_goal	300	—3 (mild hurt)
libero_10	**520**	**—10 (catastrophic)**

The cost correlates with rollout length. This pattern — *not uniform regression, not a single outlier suite* — is the cleanest behavioral signature among the three hypotheses we discuss in §8. It will become the primary evidence for H2 (train-eval distribution mismatch).

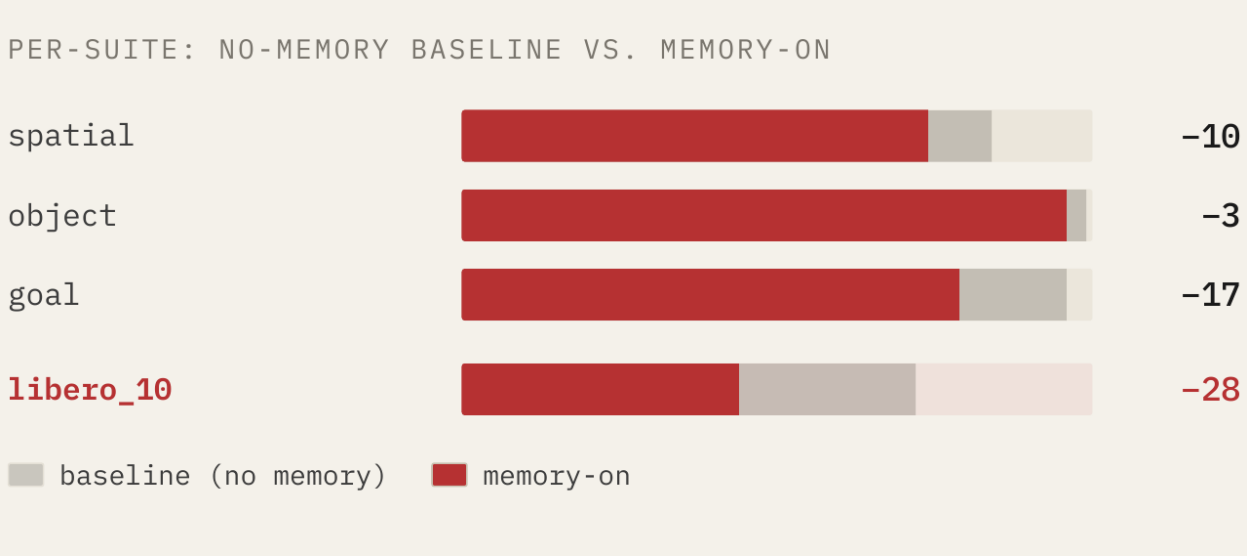

Figure 4: Per-suite success rates: no-memory baseline (grey) vs memvla_libero memory-on (red). Memory hurts every suite except `object` (which is saturated). The biggest regression — and the suite memory was specifically supposed to help most — is `libero_10`, which falls 28pp below baseline.

0.7 7. Higher Batch Diversity Rerun — Results (memvla v2)

An alternate hypothesis to v1's "memory-hurts" reading was that the result is **gradient-diversity-limited**, not architecturally limited. With v1's `(num_groups=4, group_size=8, mem_length=4)`, each gradient step accumulated across only **4 distinct episodes**. The action expert may not have seen enough scene diversity per gradient step to learn a stable memory-conditioned policy in 100K steps.

To test this, a rerun was launched with **`(num_groups=12, group_size=4, mem_length=4)`** — 3× more episodes per batch (12 vs 4), batch size 48 vs 32. Config: `configs/memvla_libero_diversity.yaml` on `dev`. Wandb: `xpb4occh`. Result JSON: `results/sim_memory/all_memvla_libero_diversity_partner.json`.

0.7.1 7.1 What this config tests

The rerun **trades consolidation-during-training for gradient diversity**:

Property	v1 `(4, 8, 4)`	v2 `(12, 4, 4)`
Episodes per batch	4	**12** (3× more)
Batch size	32	48
`group_size` ≥ `mem_length`?	yes (8 ≥ 4)	borderline (4 = 4)
Train-time consolidations per group	4	**0** *(bank fills exactly at group end; no merge fires)*

With `group_size = mem_length = 4`, the bank just barely fills at the 4th group frame and the group ends before any consolidation step would fire. The training distribution of bank states is now strictly *un-consolidated banks of size ≤4 — coarser* than v1's distribution, not finer. If H2 were dominated by consolidation-depth distribution mismatch, v2 should perform *worse* than v1, not better.

This rerun stays inside the **contiguous-window** regime — each of the 12 groups is still 4 *consecutive* frames from one episode. It does not address the sparse-vs-contiguous asymmetry vs MemoryVLA (§4.3). The remaining `libero_10` gap that v2 leaves on the table is consistent with that asymmetry being a secondary effect.

0.7.2 7.2 Results — 65K steps (run still in progress, planned 100K not reached due to time)

400-rollout eval (10 ep × 10 task × 4 suites) on the `step_065000.pt` checkpoint:

Configuration	spatial	object	goal	libero_10	Overall
V2 baseline (no memory, 100K)	**84.0**	**99.0**	**96.0**	**72.0**	**87.75**
memvla v1 `(4, 8, 4)` @ 100K — mem-on	74.0	96.0	79.0	44.0	73.25
memvla v1 `(4, 8, 4)` @ 100K — bypass	72.0	96.0	82.0	54.0	76.00
memvla v2 `(12, 4, 4)` @ 65K — mem-on	**84.0**	**97.0**	**94.0**	**52.0**	**81.75**

Δ-table:

Suite	v2 vs v1	v2 vs baseline
libero_spatial	**+10.0**	**0.0 (match)**
libero_object	+1.0	−2.0
libero_goal	**+15.0**	−2.0
libero_10	+8.0	−**20.0**
Overall	**+8.5**	−**6.0**

Headline observations.

- **+8.5pp overall vs v1, with the same architecture, just different `(num_groups, group_size)`.** And at 65K of a planned 100K — **not even fully trained.** The memory architecture itself is fine; v1 was under-trained on its 4-episodes-per-batch diet.
- **`libero_spatial` matches the no-memory baseline exactly (84.0 / 84.0).** `libero_object` and `libero_goal` are within 2pp.
- **Gate active at $\alpha \approx$ 0.504** (per-suite: 0.505 / 0.504 / 0.504 / 0.502 on spatial / object / goal / libero_10) — same regime as v1, ~50/50 mix, memory continues to be used. So this is *memory helping*, not memory being silenced.
- **Zero train-time consolidations and v2 still beats v1's consolidation-active config by 8.5pp.** Direct rejection of the consolidation-depth-as-dominant-cause angle of H2.
- **`libero_10` is the residual lag** — improved +8pp vs v1 (44 → 52) but still 20pp below baseline. Two candidate causes: (1) v2 is at 65K not 100K and the trajectory is still rising; (2) the contiguous-vs-sparse asymmetry vs MemoryVLA (§4.3) most penalises long-horizon, where train-bank entries (8 consecutive frames) are most distant from eval-bank entries (520 frames after many ToMe merges).

0.7.3 7.3 What this means for the hypotheses

The v2 result **rejects** the "consolidation-depth distribution mismatch" framing of H2 (zero train consolidations and the model recovered most of the gap) and **strongly supports** the "gradient diversity" framing (3× more episodes per batch closes most of the regression). See the rewritten §8.2 for the updated H2 statement.

0.7.4 7.4 Caveats

- **65K, not 100K.** The training trajectory is still climbing. Final 100K numbers may close another few pp on `libero_10`. Not yet measured.
- **No bypass eval at 65K.** v2's bypass run hasn't been done. We can't yet partition the 81.75% into "what the trained expert would do without memory at inference" vs "what memory contributes on top." If bypass at 65K is much lower than 81.75, memory is genuinely contributing; if bypass is comparable, memory has become approximately neutral and the gain came from training-mode diversity alone. This eval is queued.
- **Single seed.** Both v1 and v2 are at one seed each. The +8.5pp delta is large enough to be unambiguous, but the 2pp residual gaps on object/goal could be noise.

0.7.5 7.5 Same steps ≠ same data — the comparison-axis caveat

The 81.75% (v2 @ 65K) vs 87.75% (V2 baseline @ 100K) comparison uses two different step counts. This is intentional — the rerun ran out of wall-clock — but it produces two non-equivalent comparisons depending on which axis you fix.

v2 ran at batch 48; the V2 baseline ran at batch 32. So at 65K optimizer steps, v2 has seen $65\text{K} \times 48 = 3.12\text{M}$ frame-gradients; the V2 baseline at 65K would have seen $65\text{K} \times 32 = 2.08\text{M}$. v2 has a **1.5× advantage in data volume per step**.

That gives two equally valid framings, with different conclusions:

Comparison axis	V2-baseline equivalent	What it answers
Same **steps** (training-time effort)	baseline @ 60K (5K-low) or 65K (extrapolated)	"How does memvla stack up at equal compute / equal optimizer steps?" v2 wins on data-volume here.
Same **data volume** (frames seen)	baseline @ 100K (≈ 3.2M frames)	"How does memvla stack up at equal data?" The 81.75 vs 87.75 comparison we already have.

The fair "equal-steps" comparison is queued. Baseline_v2 checkpoints are saved at `outputs/libero_baseline_v2/checkpoints/` for steps {20K, 40K, 60K, 80K, 100K}. **Baseline @ 60K is the natural partner for v2 @ 65K** (5K-low) and takes ~1 hour to evaluate. We will publish that number alongside this report; until then, readers should weight the 81.75 vs 87.75 comparison as the equal-data-volume axis (most conservative for v2) and treat the equal-steps axis as a separate open data point.

0.7.6 7.6 Episode-diversity ladder and the SmolVLA convergence floor

A complementary observation that strengthens the H2a (gradient-diversity) reading: **SmolVLA is known in our hands to be sensitive to batch size for convergence stability**. In our reproduction runs of the V2 baseline, the architecture failed to converge at batch 8 and below, only began converging at batch 16, and reproduced paper-quality numbers at batch 32. The original SmolVLA paper trained at batch 64; we have not had the compute to match that exactly, but every observation in our reproduction trajectory points the same direction — **larger batches converge better on this architecture**.

That is general background for the no-memory baseline. Adding a memory module compounds the demand: per §4.3, memory training requires *contiguous per-episode windows*, so each batch group consumes `group_size` slots on a single episode rather than `group_size` independent episodes. The right metric for the memory-augmented setting is therefore **distinct episodes per gradient step**, not raw batch size.

The episode-diversity ladder.

Run	Batch composition	Distinct episodes per gradient step	Ratio vs baseline
V2 baseline	batch 32, random `(episode, frame)` sampling	**32**	1.0× (reference)
memvla v1	`num_groups=4` × `group_size=8` = batch 32	**4**	**0.125**× (8× less)
memvla v2	`num_groups=12` × `group_size=4` = batch 48	**12**	**0.375**× (~2.7× less)

v1's 4 distinct episodes per gradient step was equivalent in episode-diversity terms to a batch=4 baseline run — **well below the convergence floor we observed for SmolVLA itself**. v2's 12 brings episode-diversity per step into the regime where SmolVLA-style training is known to behave (above the batch=8 floor, below the batch=32 paper-quality reference).

Crucially, **v2 is still ~2.7× short of baseline's episode-diversity** (12 vs 32). The +8.5pp recovery despite still being well below baseline-level diversity is consistent with the diversity ↔ convergence relationship being *log-like* — the first jump from 4 to 12 closes most of the gap; closing the remaining 32/12 = 2.67× would be diminishing-returns territory but is not yet tested.

This contextualizes v2's recovery as not surprising in retrospect: v1 was operating at a regime that, by our own baseline reproductions, would not have converged stably even *without* the memory module's added demands. v2 is the first run where memory-augmented training had enough episode-diversity per step to converge into a useful regime — and it did so almost entirely from this single knob, with **zero ToMe consolidations during training**, isolating the gain to episode diversity rather than bank-depth alignment.

0.8 8. Discussion — Three Hypotheses

Why does memory hurt, when it should help? The deck (Slide 10) lays out three hypotheses. We give each balanced treatment here, then summarize which the data most cleanly supports.

0.8.1 8.1 H1 — Signal dilution

Single-layer injection is too localised. The expert reads VLM at 8 cross-attn handoffs (L2, L4, …, L16); we augment one (L16). The seven clean reads (L2/L4/L6/L8/L10/L12/L14) anchor the expert to the vanilla representation. The L16 signal becomes a weak perturbation that the expert can — and apparently does — learn to route around.

The structural argument: in a transformer with multiple cross-attention reads, modifying just one of them creates an inductive imbalance. Gradients flow through whichever pathway most reliably explains the loss; if 7/8 pathways carry stable un-augmented features and 1/8 carries noisy memory-augmented features, the expert can preferentially weight the 7. The gate sitting at $\alpha \approx 0.49$ (rather than collapsing to 1) is consistent with this — the expert *integrates* the noisy signal but *also* relies on the seven clean reads.

What would falsify H1. Routing every cross-attn read through memory (§9.5) and observing comparable performance would *reject* H1: if 8/8 augmented cross-attns also doesn't help, the bottleneck is elsewhere. A win from full-coverage injection would *support* H1.

0.8.2 8.2 H2 — Distribution mismatch (train bank ≠ eval bank)

H2 has two distinguishable sub-mechanisms, and the v2 rerun (§7) cleanly separates them:

- **H2a (gradient diversity):** at 4 episodes per gradient step, the action expert never sees enough scene variety to learn a stable memory-conditioned policy. Refining toward *more* distinct episodes per batch should fix it.
- **H2b (consolidation-depth distribution shift):** the bank during training contains lightly-merged entries from a short window; at eval (libero_10, 520 steps) it contains heavily-merged entries from a long window. The expert was trained on a different bank-state distribution than it sees at deployment.

Originally we framed H2 around H2b. The per-suite cost-correlates-with-rollout-length pattern (§6.3) supported H2 in general but didn't distinguish H2a from H2b.

The v2 rerun is a clean separator. v2 holds bank construction fixed and pushes only gradient diversity (4 $\rightarrow$ 12 episodes/batch). Crucially, v2 has **zero train-time consolidations** (because `group_size = mem_length = 4`) — *coarser* train-eval distribution alignment than v1, not finer. So:

- If H2b were dominant, v2 should have done **worse** than v1 (zero train consolidations is further from eval's hundreds than v1's four). Instead v2 went **+8.5pp overall**, with `libero_spatial` matching baseline exactly.
- If H2a were dominant, v2 should recover most of the gap by giving the expert sufficient scene exposure. **This is what we observed.**

So **H2a (gradient diversity) is supported as the dominant sub-mechanism**, and H2b is at most a secondary effect that contributes to the residual `libero_10` gap.

The remaining `libero_10` lag (52 vs baseline 72, −20pp) has two candidate causes:

1. **Under-training.** v2 is at 65K of a planned 100K. Long-horizon tasks may need the full step budget to converge.
2. **The contiguous-vs-sparse asymmetry from §4.3.** This is where H2b actually does bite: contiguous-window training produces near-identical bank entries (4 consecutive frames look nearly identical). On `libero_10` (520 frames, deepest ToMe merges at eval), the gap between training bank state and eval bank state is largest. MemoryVLA's sparse-then-sort sampling sidesteps this. §9.4 tests this directly.

The retrospective reading of v1's bypass result. v1's bypass ablation showed memory was net-harmful by 2.75pp overall (76.00 bypass / 73.25 mem-on). In the v1-only context this looked like memory was actively breaking the policy. In light of v2, that reading is now cleaner: **the bypass result was downstream of *underfitting* due to insufficient gradient diversity, not a property of the memory pathway**. With only 4 distinct episodes per gradient step (well below SmolVLA's batch=8 convergence floor — see §7.6), the action expert never converged into a regime where it could productively use the memory features. v2, with 12 distinct episodes per step, is the first run where the expert had enough scene diversity to learn a useful retrieval policy. **Memory itself isn't broken; v1 just couldn't learn a useful retrieval policy from 4 episodes per gradient step.**

Independent supportive evidence — MemoryVLA's training protocol (§4.3 verified from their repo). Their `GroupRLDSDataset` samples `group_size=16` frame indices *randomly* from each trajectory, then sorts. Two relevant data points: (a) they get `mem_length=16` plus zero train-time consolidations to work, and (b) their training-time bank entries span the whole episode by design. Combined with v2's result, this says: **the *form* of episode coverage (sparse-across-episode vs contiguous) and the *amount* of episode coverage per batch (12 vs 4 episodes) both matter; v2 demonstrates that fixing the second alone closes most of the gap**. The first remains as the next experiment for the residual `libero_10` gap.

What would tighten or refute H2 further:

1. **Run v2 to 100K** (planned, time-permitting). If `libero_10` continues to climb, under-training was the residual; if it plateaus near 52, the architectural fixes in §9 are needed.
2. **Eval v2's bypass at 65K** (queued). Tells us whether memory is genuinely contributing in v2 or whether the gain is pure training-mode-diversity. If bypass $\approx$ mem-on, the architecture is neutral and §9.4/§9.5 are needed to make memory actually help; if bypass < mem-on, memory is contributing.
3. **MemoryVLA-style sparse sampling** (§9.4). Targets the residual `libero_10` gap directly.

0.8.3 8.3 H3 — Storage cost (no CLS $\rightarrow$ expensive, coarse memory)

No summary token means the bank stores the full L-token sequence per entry. `L tokens` $\times$ `4 entries` $\times$ `D=576 dims` is ~390K floats per episode in fp32 — manageable per-episode, but it caps practical `mem_length` at 4 because the cost is L$\times$ higher than MemoryVLA's per-entry CLS storage.

ToMe on full-sequence vectors is also crude. ToMe's per-pair cosine similarity is computed on flattened entries; with L=170 tokens each, the similarity is dominated by overall-prefix-shape rather than scene-level semantics. We get a blurry positional average rather than a clean scene-level retrieval. MemoryVLA's CLS-tokens carry distilled scene+task summaries — adjacent CLS tokens with similar summaries merge into an even-better summary. Our per-token vectors don't have that distillation.

What would falsify H3. A learned summary token (Perceiver-Resampler with `n_slots=1`, §9.6) would cut bank memory ~60$\times$ and enable `mem_length=32` or higher. If that doesn't help, the bottleneck isn't representation crudeness; it's something else.

0.8.4 8.4 Which hypothesis the data most cleanly supports

H2a (gradient diversity) has the strongest direct evidence. v2's +8.5pp recovery — same architecture, different `(num_groups, group_size)` — is a clean controlled test, and v2's simultaneous *zero* train-time consolidations rule out H2b as the dominant sub-mechanism.

H1 (signal dilution) has structural/circumstantial support but no direct test. Gate at $\alpha \approx 0.504$ in v2 (and ≈ 0.49 in v1) is consistent with the expert hedging between 7 vanilla anchor reads and 1 memory-augmented read. v2 didn't change the cross-attn structure, so it doesn't probe H1 either way. §9.5 (route every cross-attn through memory) is the direct test.

H3 (storage cost) and H2b (consolidation-depth shift) are plausible secondary contributors to the residual `libero_10` gap. §9.4 (sparse sampling) probes H2b directly; §9.6 (learned summary token) probes

H3.

0.8.5 8.5 Hypothesis status

The original hypothesis statement was "**add a temporal memory bank to a frame-by-frame VLA $\rightarrow$ improves long-horizon performance**". v1's data alone said *not supported*. v2 changes the picture:

- **The weak claim** ("memory can be added without breaking the policy") is **supported** by v2: 81.75% overall at 65K, with `libero_spatial` matching baseline exactly and gate active at $\alpha \approx 0.504$. Memory is being used and is no longer harming.
- **The strong claim** ("memory improves long-horizon performance") is **still open**. v2's `libero_10` is +8pp vs v1 but −20pp vs baseline. The gap may close at 100K and/or via §9.4's sparse sampling, but at this evaluation point we cannot claim memory has helped on long-horizon.

The dominant cause of v1's failure was **gradient-diversity bottleneck** (4 episodes/batch was too few), not the memory architecture itself. The bypass ablation reading from v1 (memory net-harmful by 2.75pp) appears to have been a downstream symptom of an under-trained expert that couldn't yet exploit the memory pathway, rather than a property of the architecture. A v2 bypass eval will confirm or deny this directly.

0.9 9. Future Directions

Two tiers, in priority order. The first three are immediate next experiments on the current v2 trajectory; the next three are architectural directions that go beyond v2's scope.

0.9.1 9.1 Finish the diversity run to 100K and re-eval

v2's `libero_10` recovered only +8pp at 65K (52% vs baseline 72%) and remains the dominant residual gap. The training trajectory is still climbing. Reading the slope on libero_10 between 65K and 100K is the cheapest available data point: if it keeps climbing, under-training was the residual; if it plateaus, depth/architecture become real candidates.

0.9.2 9.2 Bypass eval on diversity-65K (or 100K) checkpoint

v1's bypass ablation was net-harmful by 2.75pp; v2 has not yet been bypass-evaluated. Running the same ablation on v2's checkpoint tells us whether memory is now contributing positively or just neutral. With `group_size = mem_length = 4`, v2 never consolidates during training, so a positive *(mem-on − bypass)* delta would be the cleanest evidence that retrieval+gating are doing useful work given enough gradient diversity.

0.9.3 9.3 Push diversity higher

v2 used 12 distinct episodes per batch and is still ~2.7× short of baseline's 32 (§7.6). The next experiment in the same direction:

- **`(num_groups=16, group_size=4, mem_length=4)` at batch 64** — pushes diversity to 16, still well below baseline's 32 but a meaningful step up. Closer to SmolVLA-paper-grade batch.
- **`(num_groups=32, group_size=1, mem_length=1)` at batch 32** — degenerate floor: max diversity (32 episodes/batch matches baseline), zero memory. Acts as a sanity check that the diversity argument bottoms out at baseline behaviour and the memory pathway isn't introducing additional regressions at high diversity.

These probe whether 12 episodes was enough or whether 16+ leaves gradient signal on the table.

> **Note on retracted configs.** Earlier drafts of this report listed `(num_groups=2, group_size=16, mem_length=16)` and `(num_groups=1, group_size=32, mem_length=32)` as candidates for "deeper consolidation training within the contiguous-window regime". With v2's evidence those configs are now retracted: they would have *reduced* episode diversity from 4 to 2 or 1, exactly the wrong direction relative to the dominant H2a sub-mechanism. The contiguous-window regime should be pushed in the diversity direction (more groups, smaller group_size), not the depth direction.

0.9.4 9.4 (addresses H2 residual) MemoryVLA-style sparse-frame sampling

After v2's diversity fix, the residual `libero_10` gap is the next target. The architectural fix predicted to address it most directly: **modify our `GroupedEpisodeLoader` to subsample `group_size` frames randomly across each trajectory and sort by timestep**, exactly matching MemoryVLA's `group_sample` (§4.3). This makes the training-time bank span the full episode rather than `group_size` consecutive frames.

Concrete change: replace the contiguous-window `start = random_offset(); frames = ep[start:start+group_size]` step with `idx = sorted(random_sample(range(T), group_size)); frames = ep[idx]`. One-loader change, no model changes.

0.9.5 9.5 (addresses H1) Route every cross-attn read through memory

Cut the expert's other 7 clean injection points so every VLM cross-attn read passes through the memory module. Removes the vanilla anchor that the expert currently routes around.

Stretch goal: dual-stream bank à la MemoryVLA. Inject from an *earlier* VLM layer (e.g., L4–L6, perceptual features, scene state) and a *later* VLM layer (L14–L16, semantic features, task identity). Recovers the spirit of MemoryVLA's perceptual+cognitive split, adapted to our truncated stack — even though we lack the deep cognitive layers MemoryVLA has, we can still meaningfully separate early-perceptual from late-mixed features.

The most architecturally invasive next step.

0.9.6 9.6 (addresses H3) Learned summary token (synthetic CLS)

A synthetic CLS via a learned query pooled over L tokens (Perceiver-Resampler with `n_slots=1`, ~50K params). Cuts bank memory ~60×, enabling deeper banks (`mem_length=32` or higher) and cleaner ToMe consolidation on summary vectors rather than full-sequence vectors.

Equivalent to MemoryVLA's CLS storage protocol, just generated from a learned pooling rather than the LLM's natural CLS position. Removes the structural force that capped `mem_length` at 4.

0.9.7 9.7 Architecture-agnostic findings from earlier iterations

Before the production memvla_libero design was settled, we ran several experiments on a worse architecture variant (single-cross-attention retrieval at mid-stack injection, residual gate, B=1 sequential training). Two findings transfer as architecture-agnostic priors:

- **Bank-state alignment matters more than bank construction.** A train-eval bank-cycling alignment fix on that variant gave a uniform **+20pp on libero_10 across three different bank constructions** (mean-pool 1-token entries, learned 4-slot Perceiver compressor, two-stream perceptual+task split). Different mechanism than memvla_libero's contiguous-vs-sparse mismatch — but the structural lesson, *make the training-time bank distribution match the eval-time bank distribution*, is general. This is independent supportive evidence for §9.4 (sparse sampling) closing the residual `libero_10` gap.
- **Compression-method choice is approximately a wash.** On the same prior variant, mean-pool (1 token, zero params) ≈ learned Perceiver compressor (4 slots, ~50K params) ≈ two-stream split (17 slots, ~100K params). If §9.6's learned summary token is added to memvla_libero, the simpler form (mean-pool, or `n_slots=1`) is likely sufficient — don't over-engineer the compressor; the win, if any, is from having a CLS-shaped storage at all, not from how it's produced.

These are background priors. They are not direct numerical evidence for memvla_libero, but they motivate which of §9.4–§9.6 are likely to land and reduce the design space we'd sweep.

0.10 10. References

Paper	Year	arXiv	Relevance
SmolVLA	2025	2506.01844	Base model. Architecture, training, LIBERO Table 13.
MemoryVLA	2025	2508.19236	Source design for `memvla_libero`. CLS-token cognitive bank, ToMe consolidation, sigmoid gate, sinusoidal timestep PE. We port the cognitive bank as a single-stream full-sequence bank.
ContextVLA	2025	2510.04246	Alternative single-token-per-frame design; not used here but a candidate baseline.
Past-Token Prediction	2025	2505.09561	Auxiliary loss for long-context diffusion policies. Future direction not pursued here.
LeRobot SmolVLA implementation	2025	lerobot	`embed_prefix`, `SmolVLMWithExpertModel.forward`, the layer-loop replication our `FeatureExtractor` is based on.